

# iOS面试题合集（数据结构篇）

iOSer iOSer 5月19日

收录于话题

#BAT大厂面试题合集

9个

欢迎点击上方蓝字“iOSer”关注我们  
再点击右上角“...”菜单，选择“设为星标”  
我们一起精进、成长！

本栏目将持续更新--请iOS的小伙伴关注我们!

做这个的初心是希望能巩固自己的基础知识，

当然也希望能帮助更多的开发者，

可以加入我们的技术交流群：**834688868**，群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻，分享BAT，阿里面试题、面试经验，讨论技术，大家一起交流学习成长！

## 目录：

- 1.数据结构的存储一般常用的有几种？各有什么特点？
- 2.集合结构 线性结构 树形结构 图形结构
- 3.单向链表 双向链表 循环链表 分别为什么？
- 4.数组和链表区别有哪些？
- 5.堆、栈和队列 分别是什么？
- 6.输入一棵二叉树的根结点，求该树的深度？
- 7.输入一棵二叉树的根结点，判断该树是不是平衡二叉树？
- 8.字符匹配 & 字符去重的方法

## 1.数据结构的存储一般常用的有几种？各有什么特点？

数据结构的存储一般常用的有两种 顺序存储结构 和 链式存储结构

- 顺序存储结构:

比如，数组，1-2-3-4-5-6-7-8-9-10，存储是按顺序的。再比如栈和队列等

- 链式存储结构:

比如，数组，1-2-3-4-5-6-7-8-9-10，链式存储就不一样了 1(地址)-2(地址)-7(地址)-4(地址)-5(地址)-9(地址)-8(地址)-3(地址)-6(地址)-10(地址)。每个数字后面跟着一个地址 而且存储形式不再是顺序

## 2.集合结构 线性结构 树形结构 图形结构

- 集合结构

一个集合，就是一个圆圈中有很多个元素，元素与元素之间没有任何关系 这个很简单

- 线性结构

一个条线上站着很多个人。这条线不一定是直的。也可以是弯的。也可以是值的 相当于一条线被分成了好几段的样子（发挥你的想象力）。线性结构是一对一的关系

- 树形结构

做开发的肯定或多或少的知道xml 解析 树形结构跟他非常类似。也可以想象成一个金字塔。树形结构是一对多的关系

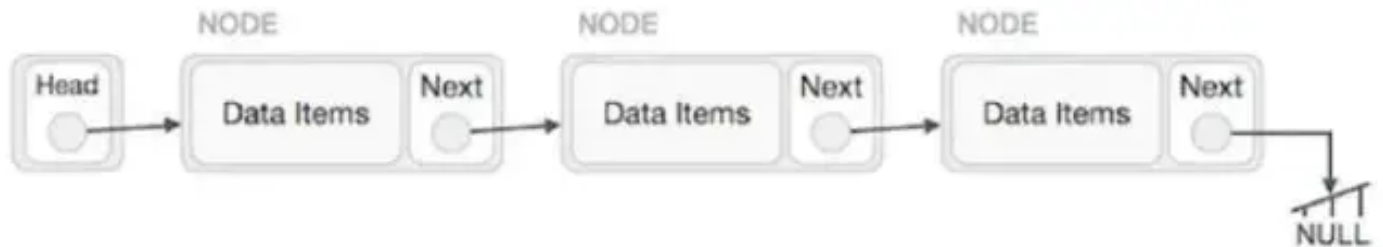
- 图形结构

这个就比较复杂了。他呢 无穷。无边 无向（没有方向）图形机构 你可以理解为多对多 类似于我们人的交集关系

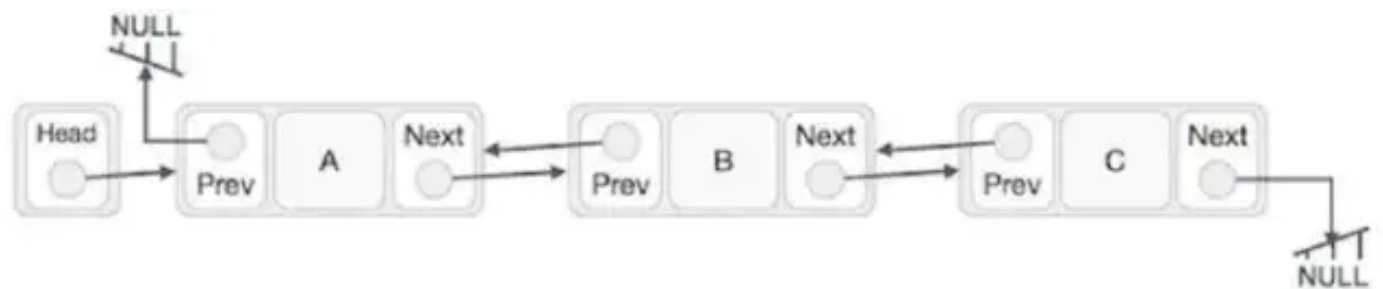
### 3.单向链表 双向链表 循环链表

#### 单向链表

A->B->C->D->E->F->G->H. 这就是单向链表 H 是头 A 是尾 像一个只有一个头的火车一样只能一个头拉着跑



#### 双向链表



#### 循环链表

循环链表是与单向链表一样，是一种链式的存储结构，所不同的是，循环链表的最后一个结点的指针是指向该循环链表的第一个结点或者表头结点，从而构成一个环形的链。发挥想象力 A->B->C->D->E->F->G->H->A. 绕成一个圈。就像蛇吃自己的这就是循环 不需要去死记硬背哪些理论知识。

### 4.数组和链表的区别

#### 数组

数组元素在内存上连续存放，可以通过下标查找元素；插入、删除需要移动大量元素，比较适用于元素很少变化的情况

## 链表

链表中的元素在内存中不是顺序存储的，查找慢，插入、删除只需要对元素指针重新赋值，效率高

## 5.堆、栈和队列 分别是什么？

### 堆

- 堆是一种经过排序的树形数据结构，每个节点都有一个值，通常我们所说的堆的数据结构是指二叉树。所以堆在数据结构中通常可以被看做是一棵树的数组对象。而且堆需要满足一下两个性质：
  - 1) 堆中某个节点的值总是不大于或不小于其父节点的值；
  - 2) 堆总是一棵完全二叉树。
- 堆分为两种情况，有最大堆和最小堆。将根节点最大的堆叫做最大堆或大根堆，根节点最小的堆叫做最小堆或小根堆，在一个摆放好元素的最小堆中，父结点中的元素一定比子结点的元素要小，但对于左右结点的大小则没有规定谁大谁小。
- 堆常用来实现优先队列，堆的存取是随意的，这就如同我们在图书馆的书架上取书，虽然书的摆放是有顺序的，但是我们想取任意一本时不必像栈一样，先取出前面所有的书，书架这种机制不同于箱子，我们可以直接取出我们想要的书。

### 栈

- 栈是限定仅在表尾进行插入和删除操作的线性表。我们把允许插入和删除的一端称为栈顶，另一端称为栈底，不含任何数据元素的栈称为空栈。栈的特殊之处在于它限制了这

个线性表的插入和删除位置，它始终只在栈顶进行。

- 栈是一种具有后进先出的数据结构，又称为后进先出的线性表，简称 LIFO（Last In First Out）结构。也就是说后存放的先取，先存放的后取，这就类似于我们要在取放在箱子底部的东西（放进去比较早的物体），我们首先要移开压在它上面的物体（放进去比较晚的物体）。
- 堆栈中定义了一些操作。两个最重要的是PUSH和POP。PUSH操作在堆栈的顶部加入一个元素。POP操作相反，在堆栈顶部移去一个元素，并将堆栈的大小减一。
- 栈的应用—递归

## 队列

- 队列是只允许在一端进行插入操作、而在另一端进行删除操作的线性表。允许插入的一端称为队尾，允许删除的一端称为队头。它是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作，和栈一样，队列是一种操作受限制的线性表。
- 队列是一种先进先出的数据结构，又称为先进先出的线性表，简称 FIFO（First In First Out）结构。也就是说先放的先取，后放的后取，就如同行李过安检的时候，先放进去的行李在另一端总是先出来，后放入的行李会在最后面出来。

## 6.输入一棵二叉树的根结点，求该树的深度？

二叉树的结点定义如下：



```
struct BinaryTreeNode
{
    int m_nValue ;
```

```
    BinaryTreeNode* m_pLeft;  
    BinaryTreeNode* m_pRight ;  
}
```

- 如果一棵树只有一个结点，它的深度为1。
- 如果根结点只有左子树而没有右子树，那么树的深度应该是其左子树的深度加1；同样，如果根结点只有右子树而没有左子树，那么树的深度应该是其右子树的深度加1。
- 如果既有右子树又有左子树，那该树的深度就是其左、右子树深度的较大值再加1。



```
int TreeDepth(TreeNode* pRoot)  
{  
    if(pRoot == nullptr)  
        return 0;  
    int left = TreeDepth(pRoot->left);  
    int right = TreeDepth(pRoot->right);  
  
    return (left>right) ? (left+1) : (right+1);  
}
```

## 7.输入一棵二叉树的根结点，判断该树是不是平衡二叉树？

- 重复遍历结点

先求出根结点的左右子树的深度，然后判断它们的深度相差不超过1，如果否，则不是一棵二叉树；如果是，再用同样的方法分别判断左子树和右子树是否为平衡二叉树，如果都是，则这就是一棵平衡二叉树。

- 遍历一遍结点

遍历结点的同时记录下该结点的深度，避免重复访问。

## 方法1:



```
struct TreeNode{
    int val;
    TreeNode* left;
    TreeNode* right;
};

int TreeDepth(TreeNode* pRoot){
    if(pRoot==NULL)
        return 0;
    int left=TreeDepth(pRoot->left);
    int right=TreeDepth(pRoot->right);
    return left>right?(left+1):(right+1);
}

bool IsBalanced(TreeNode* pRoot){
    if(pRoot==NULL)
        return true;
    int left=TreeDepth(pRoot->left);
    int right=TreeDepth(pRoot->right);
    int diff=left-right;
    if(diff>1 || diff<-1)
        return false;
    return IsBalanced(pRoot->left) && IsBalanced(pRoot->right)
;
}
```

## 方法2:



```
bool IsBalanced_1(TreeNode* pRoot,int& depth){
    if(pRoot==NULL){
        depth=0;
        return true;
    }
}
```

```
    }
    int left,right;
    int diff;
    if(IsBalanced_1(pRoot->left,left) && IsBalanced_1(pRoot->right,right)){
        diff=left-right;
        if(diff<=1 || diff>=-1){
            depth=left>right?left+1:right+1;
            return true;
        }
    }
    return false;
}

bool IsBalancedTree(TreeNode* pRoot){
    int depth=0;
    return IsBalanced_1(pRoot,depth);
}
```

## 8.字符匹配 & 字符去重

### 题目

给你一个仅包含小写字母的字符串，请你去除字符串中重复的字母，使得每个字母只出现一次。需保证返回结果的字典序最小（要求不能打乱其他字符的相对位置）



示例1：

输入："bcabc"

输出："abc"

示例2：

输入："cbacdcbc"

输出："acdb"



## 题目分析

题目的意思,你去除重复字母后,需要按最小的字典序返回.并且不能打乱其他字母的相对位置

字典序:

字符串之间比较和数字比较不一样,字符串比较是从头往后挨个字符比较,那个字符串大取决于两个字符串中第一个对应不相等的字符;

部分总结的数据结构面试题就到这里了,  
以上内容对你有帮助的话,记得关注我们!

加入我们的iOS高级技术交流QQ群: **834688868**, 群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻, 分享BAT, 阿里面试题、面试经验, 讨论技术, 大家一起交流学习成长!

文章来源于网络, 已尽可能标明作者以及来源, 文章内容为作者独立观点, 不代表本公众号立场, 因文章侵权本公众号不承担任何法律及连带责任, 如有侵权, 请联系我们删除。



觉得不错的话，别忘了点个"在看"哦~ 📌

收录于话题 #BAT大厂面试题合集 · 9个

上一篇 · iOS面试题合集（算法篇）

喜欢此内容的人还喜欢

iOS技术人员如何做晋升答辩？

iOSer

新冠病毒疫苗接种情况

健康中国